The Ledger: Addendum to v10.3

StatesHome — Where Does Unit State Live?
A resolution of §7 by 27-iteration adversarial multi-agent review

R. Delloye

April 2026 — Addendum A1 to v10.3

Abstract

This addendum resolves the design question raised in §7 of the Ledger v10.3 specification: *should the state of a unit be attached to the wallet or to the unit?* Four concrete test cases — a future whose state depends on past transactions, a managed account whose state is inherently wallet-attached, a QIS strategy that trades futures, and an instrument that exists in the universe but has not been traded yet — force the design. The conclusion, reached after twenty-seven independent adversarial agent invocations across eight review rounds, is that unit state lives in three distinct maps: an immutable versioned *ProductTerms*, a shared mutable *UnitStatus*, and a per-position *PositionState*. There is no separate wallet-keyed state sector, because every economic per-wallet fact is naturally keyed by a mandate- or strategy-unit and collapses into *PositionState*[w, u_{MA}]. Twelve conditions, denoted C1–C12, make seven of the ten core invariants of §11 structurally unreachable. This addendum supersedes the phrasing at v10.3 line 1034 and line 2287.

Contents

1 The Question **3**

2 The Ruling **3**
2.1 The Three Keys, in One Line Each . . . 3
2.2 Two Orthogonal Disciplines on *PositionState* . . . 3
2.3 Conservation Lives at the Event Handler, Not at the Storage Shape . . . 4
2.4 The Twelve Conditions C1–C12 . . . 4

3 Effect on v10.3, Line 1034 **5**

4 Application to the Four Test Cases **5**
4.1 Future with State Depending on Past Transactions . . . 5
4.2 Managed Account — The "Inherently Wallet-Attached" Case . . . 6
4.3 QIS Strategy that Trades Futures . . . 6
4.4 Instrument in the Universe that Has Not Been Traded Yet . . . 7

5 Pareto Frontier — Alternatives Considered and Rejected **7**
5.1 Why Exactly Three Maps — Not Two, Not Four . . . 8

6 Invariants that Become Structurally Unreachable **9**

7 Mutation-Score and Testing Expectations **9**

8 Risk Register — Institutional-Brake Gate (R8) **9**

9 Iteration Log **10**

10 Minimal Reference Implementation (≤ 50 lines) **11**

11 The One-Sentence Answer **12**

1 The Question

> Should the state of a unit be attached to the wallet or to the unit?
>
> Consider: (1) a future whose state depends on past transactions; (2) a managed account, whose state is inherently attached to a wallet; (3) a QIS strategy that trades futures; (4) an instrument that exists in the universe but has not been traded yet.

Adversarial reviewers — `formalis`, `jane-street-cto`, `testcommittee`, `correctness-architect`, `institutional-brake` — stress-tested every proposal on testability, correctness, and simplicity.

2 The Ruling

State attaches to three distinct objects, each with its own map and its own totality discipline.

```
ProductTerms  : Map[UnitId, NonEmptyList[TermsVersion]]   # total on registered u,
    append-only, versioned
UnitStatus    : Map[UnitId, UnitStatus]                   # total on registered u,
    mutable, shared across holders
PositionState : Map[(WalletId, UnitId), PositionState]    # monotone carrier,
   Option accessor, per-(holder, unit)
```

Plus a non-state, non-financial sidecar:

```
WalletRegistry : Map[WalletId, WalletMetadata]            # KYC, permissions,
   audit cursor, NOT state
```

There is no `WalletState` sector. Every economic per-wallet fact turned out to be (w, u_{mandate})-keyed once the mandate/strategy is given its rightful identity as a unit — so the W-sector is empty of economic state and collapses into *PositionState*.

2.1 The Three Keys, in One Line Each

Key	Home	Examples
u (immutable)	*ProductTerms*	multiplier, currency, expiry, CCP, strike, ISIN, fee schedule, mandate text, benchmark identity, index methodology
u (mutable, shared)	*UnitStatus*	`lifecycle_stage`, `last_settlement_price`, `last_settlement_date`, `current_weights`, `nav_index`, `triggered_barrier`, `superseded_by`
(w, u)	*PositionState*	`accumulated_cost`, `ccp_binding`, per-position OTC lifecycle, `entry_nav`, `hwm`, `accrued_{mgmt,perf}_fee`, `benchmark_nav_at_inception`, `mandate_breach_flags`

2.2 Two Orthogonal Disciplines on *PositionState*

- **Accessor returns `Option[PositionState]`** (Lattner, Jane Street): `None` means *this wallet has never held this unit.* `Some(zero)` means *held once, currently flat.* Both readings are load-

bearing (VM-settle, wash-sale lookback, record-date entitlements); `None` versus `Some(zero)` cannot be collapsed.

- **Carrier is monotone** (Feynman, testcommittee): once created, a row is never garbage-collected. Close-out leaves a `zero` row. This makes replay a literal fold — `apply_all(events)` is identical regardless of checkpoint boundaries — and makes the conservation property a single pass over a stable key set.

The two are orthogonal: one is type discipline, the other is storage discipline. Both are required. This is **C1**.

2.3 Conservation Lives at the Event Handler, Not at the Storage Shape

No map layout can enforce $\sum_w \texttt{accumulated\_cost}(w, u) = 0$ by types alone; refinement types on decimal sums are not free in any production language. Instead, every event handler — `Trade`, `SettleVM`, `CorporateAction`, `QISRebalance`, `MandateAmend` — produces a `StateDelta` that satisfies

$$\sum_w \Delta f(w, u) = 0$$

structurally per event class, with explicit proofs for the 2-leg, K-leg, VM-fan-out, and vacuous (zero-holder) base cases. Induction over the event stream gives the global invariant. This is **C2**.

2.4 The Twelve Conditions C1–C12

C1 Option accessor + monotone carrier, both.

C2 Handler-level conservation, per event class, with vacuous base case.

C3 Atomic `StateDelta` across *ProductTerms* / *UnitStatus* / *PositionState*. Partial application rejected.

C4 Capability-scoped reads; cross-(w, u_{MA}) overlay reads forbidden; strategy exports flow only through *UnitStatus*.

C5 *UnitStatus* registration-total; product-declared defaults applied at Unit Store registration.

C6 *ProductTerms* versioned append-only; no in-place mutation.

C7 *ProductTerms* registration-total; first write at registration.

C8 Amendment two-track: a product-declared fungibility predicate decides *Preserving* (append `TermsVersion`) versus *Breaking* (allocate fresh u + `SupersededBy`).

C9 Handlers on zero-holder units discharge $\sum = 0$ vacuously; per-class proof obligation includes the empty case.

C10 Re-registration of a u_{id} is a hard error — never a silent reset.

C11 Each *PositionState* field is tagged with the unique handler allowed to mutate it (`ac` → settle/trade; `hwm` → fee_crystallise; `entry_nav` → subscribe; ...).

C12 All per-$(w, \text{mandate/strategy})$ economic state lives at *PositionState*$[w, u_{\text{MA}}]$ / $[w, u_{\text{QIS}}]$ — no flat per-wallet scalars for economic state. W-sector collapse enforced by schema.

3 Effect on v10.3, Line 1034

The v10.3 main document currently reads:

> *"Unit state is per-unit for most instrument types. For instruments with per-wallet state (e.g., futures accumulated cost), the state dictionary is per (wallet, unit) pair."*

This phrasing is replaced by:

> Every unit $u \in \mathcal{U}$ carries immutable *ProductTerms*$[u]$ (versioned append-only) and mutable *UnitStatus*$[u]$ (shared across all holders). Every held position (w, u) carries a *PositionState*$[w, u]$, with `None` on the accessor distinguishing "never held" from `Some`$(zero_P)$ "held-and-flat". State pertaining to a holder's relationship to a strategy, mandate, or managed account lives at *PositionState*$[w, u_{\text{MA}}]$ where u_{MA} is the mandate/strategy contract unit. There is no separate wallet-keyed state sector.

The change is additive-then-swap: `get_unit_state`(u) remains as a deprecated alias to `product_terms`(u) `++` `unit_status`(u); `get_unit_state`(w, u) at line 2287 maps to `position_state`(w, u).

4 Application to the Four Test Cases

4.1 Future with State Depending on Past Transactions

Field	Home	Rationale
`multiplier`, `currency`, `expiry`, `clearinghouse`, `exchange`, `product_id`	*ProductTerms*$[u_{\text{ES}}]$	Immutable. CME-ES and ICE-ES are *distinct units* (this replaces v10.3 line 1168's "per-wallet clearinghouse" phrasing).
`lifecycle_stage`, `last_settlement_price`, `last_settlement_date`	*UnitStatus*$[u_{\text{ES}}]$	Shared across all holders. One settle price per contract.
`accumulated_cost` (`ac`)	*PositionState*$[w, u_{\text{ES}}]$	Conservation $\sum_w \texttt{ac}(w, u) = 0$ holds by handler-level structural zero-sum on every `Trade` (buyer delta + seller delta = 0) and every `SettleVM` (each wallet reset to target; cash moves offset exactly).

- **Per-(w, u) state earns its place** by the Karpathy substitution test: two wallets holding the same contract can have different `ac`; any u-keyed layout would collapse them.
- `first_touch_date` **is NOT state** — it is derived from the event log on demand. Caching it in *PositionState* would create a fold-inconsistency under back-dated corrections (R5_futures A3).
- **Settled positions retain their rows** (monotone carrier). Tax reporting, wash-sale lookback, and 1099-B reconstruction demand the ghost row.

4.2 Managed Account — The "Inherently Wallet-Attached" Case

The mandate itself is a unit (u_{MA}). v10.3 §6 already places mandates in $\mathcal{U}$ ("the managed-account smart contract", "CSA margin as a wallet-level smart contract"). The mandate is issued by the manager and held by the client:

$$w_{\text{manager}}(u_{\text{MA}}) = -1 \qquad w_{\text{client}}(u_{\text{MA}}) = +1 \qquad \sum_{w} w(u_{\text{MA}}) = 0.$$

With u_{MA} promoted to a first-class unit, every field that *looked* per-wallet in Sec 6 relocates naturally:

Field	Home
Mandate text, fee schedule, benchmark identity, max position limits	*ProductTerms*$[u_{\text{MA}}]$
HWM hurdle methodology, crystallisation frequency	*ProductTerms*$[u_{\text{MA}}]$
HWM *value* (client-specific), `hwm_date`	*PositionState*$[w_{\text{client}}, u_{\text{MA}}]$
Accrued mgmt / perf fee, mandate breach flags, subscription / redemption cursor	*PositionState*$[w_{\text{client}}, u_{\text{MA}}]$
Benchmark NAV at this wallet's inception	*PositionState*$[w_{\text{client}}, u_{\text{MA}}]$
Current benchmark level (shared, from index source)	*UnitStatus*$[u_{\text{bench}}]$

Why this is not the Dirac $u_\emptyset$ hack. $u_\emptyset$ had no issuer, no holder, no conservation partner — it was a self-reflexive sentinel that broke $\sum_w = 0$ by fiat. u_{MA} is a real contract with a real issuer (manager) and a real holder (client); conservation holds by the standard issuance law.

Multi-mandate composition (a client with base + overlay mandate on the same wallet) is handled natively: two separate $(w_{\text{client}}, u_{\text{MA,base}})$ and $(w_{\text{client}}, u_{\text{MA,overlay}})$ rows, each with its own HWM, fees, breach flags. A flat per-wallet scalar would have collapsed them.

4.3 QIS Strategy that Trades Futures

Five distinct keyings coexist — and no two are in the same map.

```
u_QIS         -- the strategy itself (a tradable unit)
u_ES, u_NQ    -- the futures the strategy trades
u_MA_C        -- the mandate under which client C holds QIS (if applicable)
w_QIS         -- the strategy's own wallet (where futures positions live)
w_C           -- a client wallet
```

State	Home
Strategy contract terms (vol target, barrier, universe, share-class index start)	*ProductTerms*$[u_{\text{QIS}}]$
Strategy current state (`last_rebalance_date`, `current_weights`, `cumulative_return`, `nav_index`, `vol_realised`, `triggered_barrier`)	*UnitStatus*$[u_{\text{QIS}}]$ — *strategy-level, shared across all holders*
Futures positions held by the strategy (`accumulated_cost`, ccp)	*PositionState*$[w_{\text{QIS}}, u_{\text{ES}}]$, $[w_{\text{QIS}}, u_{\text{NQ}}]$, $[w_{\text{QIS}}, u_{\text{YM}}]$
Per-client subscription state (`entry_nav`, `hwm`)	*PositionState*$[w_C, u_{\text{QIS}}]$

Per-client mandate overlay (if any)	$PositionState[w_C, u_{\text{MA,C}}]$

HWM resolved. R5 had a live tension: $WalletState[w_C]$.overlays$[u_{\text{MA}}]$ (R5_managed_account) versus $PositionState[w_C, u_{\text{QIS}}]$ (R5_qis). `correctness-architect` and `formalis` broke the tie: a client holding *two* strategies must carry *two* HWMs, so any w-only keying collapses them. HWM lives at $PositionState[w_C, u_{\text{QIS}}]$ (or $[w_C, u_{\text{MA,C}}]$ when a mandate wrapper is in force).

Rebalance event. The barrier-hit or monthly-rebalance event is one atomic `StateDelta` that updates $UnitStatus[u_{\text{QIS}}]$ (new weights, new lifecycle if barrier breached) *and* the strategy's $PositionState[w_{\text{QIS}}, u_{\text{ES/NQ/YM}}]$ rows (via trade moves on the underlying futures). C3 mandates atomicity across all three maps.

Wind-down. $UnitStatus[u_{\text{QIS}}]$.`lifecycle_stage` → `CLOSED` propagates to clients as a NAV-strike redemption event. $PositionState[w_C, u_{\text{QIS}}]$ rows are **retained** with balance 0 (monotone carrier); they preserve the audit trail and the final HWM for tax reporting.

4.4 Instrument in the Universe that Has Not Been Traded Yet

- $ProductTerms[u]$ — **present** at Unit Store registration. C7.
- $UnitStatus[u]$ — **present and fully initialised** at registration with product-declared defaults (e.g., `lifecycle_stage = LISTED`, `last_settlement_price = None`). C5. `view.unit_status`(u) is *total*.
- $PositionState[w, u]$ — **no rows** for any w. `view.position_state`$(w, u) =$ `None` for all w until first touch. C1.
- $WalletRegistry[w]$ — unaffected.

Lifecycle totality. An untraded option can transition `LISTED` → `ACTIVE` → `EXPIRED` purely through *UnitStatus* updates, with zero *PositionState* rows ever created. Handlers fire on `holders_of`$(u) = \emptyset$ and discharge $\sum_w \Delta = 0$ vacuously (C9). A naive bug class — `dividend_per_share / len(holders)` — is caught by C9's explicit empty-set obligation.

Amendment discipline. A bond amendment that preserves fungibility (coupon step-up, CSA eligible-collateral change, fee-rate tweak within declared band) appends a `TermsVersion` to *ProductTerms*$[u]$ — existing *PositionState* rows survive untouched. An amendment that breaks fungibility (new ISIN, benchmark swap, bond restructuring) allocates a fresh u_{new}, stamps `SupersededBy`$(u_{\text{old}} \to u_{\text{new}})$ in both *UnitStatus* rows, and emits an atomic re-subscription `StateDelta` that moves $(w, u_{\text{old}}) \to (w, u_{\text{new}})$ preserving $\sum_w$. The product-declared fungibility predicate

`is_fungibility_preserving` : $ProductTerms \times TermsAmendment \to$ {`Preserving`, `Breaking`}

is total and testable. C8.

5 Pareto Frontier — Alternatives Considered and Rejected

The iteration tested six candidates on three axes (0–10, higher = better; simplicity axis inverted for display):

	Testability	Correctness	Simplicity (viable)
A — v10.3 current (per-unit + per-(w,u) for futures)	4	5	4
B — 3 maps + C1–C12 (ship)	**9**	**9**	**8**
C — Dirac $\sigma : W \times U \rightharpoonup S_u$ with $u_\emptyset$ sentinel	7	3	7
D — Minsky 4-map (retains explicit *WalletState*)	7	7	5
E — Grothendieck sheaf on $H_t \subseteq W \times U$	8	9	2
F — 2-map (*ProductTerms* + *PositionState*) with universe wallet $w_\star$	5	6	6

Domination findings.

- **B strictly dominates A, C, D, F** on all three axes.
- **B dominates E** (Grothendieck): equal on correctness, higher on testability (sheaf generators require non-existent libraries), radically higher on simplicity. Elegance is real; shippability is not.
- **F looks simpler but isn't.** Folding *UnitStatus* into *PositionState*$[w_\star, u]$ re-introduces the per-unit/per-(w,u) split as a *runtime predicate* `is_universe`(w) on the wallet axis. Conservation $\sum_w \Delta\mathtt{ac}(w,u) = 0$ must then *exclude* $w_\star$ explicitly — a carve-out, not a rule. This is the Minsky denormalisation trap. **B's *UnitStatus* is the type-level expression of what F hides as a wallet-id convention.**
- **C (Dirac $u_\emptyset$)** fails day-one correctness: $u_\emptyset$ has no issuer, no conservation partner. It breaks $\sum_w w(u) = 0$ by fiat.

B is the unique Pareto-optimum under any correctness gate ≥ 7. The iteration has converged.

5.1 Why Exactly Three Maps — Not Two, Not Four

Three independent forcing constraints, each of which breaks a different map:

1. **Karpathy substitution** — two wallets holding the "same" contract need distinct lifecycle state $\Rightarrow$ *PositionState*$[w,u]$ is required.
2. **Shared observables** — `last_settlement_price`, index values, strategy weights are one-per-contract observables dereferenced by every holder $\Rightarrow$ a u-keyed map distinct from per-holder state is required.
3. **Append-only versioned terms vs mutable per-u status** — *ProductTerms* is never mutated in place (audit, amendment trail, regulatory reconstruction); *UnitStatus* is written on every settle. Merging them would conflate two mutation disciplines under one name $\Rightarrow$ a third map is required.

Removing any of the three breaks one of (i)–(iii). Adding a fourth (explicit *WalletState*) introduces a sector whose economic content is empty (R6_formalis §2 enumerates every candidate and shows each collapses to (w, u_{MA})). The three-map schema is the minimum basis of the problem, not a compromise.

6 Invariants that Become Structurally Unreachable

Under B with C1–C12, 7 of v10.3's 10 core invariants (§11) are **structurally unreachable** — the system cannot express the illegal state:

P1 (conservation of quantity) — enforced by handler-level per-event-class structural zero-sum (C2) and atomic `StateDelta` (C3).

P3 (determinism of replay) — monotone carrier (C1) makes `apply_all(events[:` k`])` `++` `events[`k` :]` $\equiv$ `apply_all(events)` a literal fold identity.

P5 (idempotency of lifecycle events) — single (w, u) lattice and per-field canonical handler (C11) make idempotency a per-key dedup, not a cross-map coordination.

P6 (immutability of terms) — `NonEmptyList[TermsVersion]` append-only (C6) + registration totality (C7).

P7 (no in-place mutation of identity) — C10 (re-registration rejected) + C8 (breaking amendments allocate u_{new}, never rewrite u_{old}).

P9 (capability scoping) — C4 prohibits cross-(w, u_{MA}) overlay reads.

P10 (handler-field canon) — C11 names the unique writer per field; mutation by any other handler is a type error.

No other candidate exceeds 3 structurally-unreachable invariants (R7_correctness).

7 Mutation-Score and Testing Expectations

The `testcommittee` (R4, R7) committed to concrete numbers:

- Event handlers (arithmetic on `Decimal`): **85–90%** mutation score — conservation kills most sign/coefficient mutants.
- Lifecycle guards (`LISTED` → `ACTIVE` → `EXPIRED`): **70–80%** — boundary-comparison mutants (`>` vs `>=`) require targeted tests.
- Overall core: $\geq$ **80%** — meets the Feathers change-safety threshold.
- State-machine spec at $|W| = 3$, $|U| = 2$, depth ≤ 6: 10^5–10^6 reachable states (TLC-tractable); conservation-violating handlers caught at depth 1.

The generator universe — CDM enum × product-type cross-product — is finite and enumerable, so test coverage is structurally bounded.

8 Risk Register — Institutional-Brake Gate (R8)

The `institutional-brake` reviewer raised eight concerns. None invalidate the design on correctness grounds; all are migration / governance / operational risks that must be budgeted for.

#	Risk	Mitigation

F1	Migration scope of retrofitting v10.3 (every downstream reader of `unit_state`)	Ship `get_unit_state` as deprecated alias; additive split before cutover; parallel-run at least one quarter with reconciliation.
F2	Ownership of the C8 fungibility predicate is ungoverned	Assign per-product RACI in the Unit Store registry; Legal/Product/Risk sign-off required for predicate changes; version the predicate itself.
F3	Monotone carrier key-space growth at $10^7 \times 10^6$ scale is unbenchmarked	Benchmark Option-accessor cost and cache behaviour before merge; add tombstone-compaction policy for *PositionState* where a row has been `Some(zero)` for $>$ retention horizon.
F4	C1–C12 is a multi-sprint test programme, not a release	Land the schema first with C1/C5/C6/C7/C9/C10 (type-enforced-for-free); stage C2/C3/C8/C11 over subsequent releases; never merge without at least C1 + C2 + C3.
F5	Mandate-as-unit creates an SFTR/EMIR reporting surface for mandate issuance	Pre-flight with Regulatory team; determine whether u_{MA} issuance requires UTI / LEI pair; may need opt-out flag on *ProductTerms*$[u_{\mathrm{MA}}]$.`reportable`.
F6	CDM alignment (`TradeState` per `Trade` vs *PositionState*$[w, u]$) is asserted, not verified	Rerun Rosetta NS1–7 mapping against the 3-map schema before any CDM adapter work; publish a delta document.
F7	Onboarding cost of three maps + Option/Monotone + 12 conditions is real	Ship a decision tree and a runbook; mandatory training for new engineers before they touch handlers; one-page cheat sheet.
F8	Forward migration is not reversible once *WalletState* is collapsed	Keep a read-only mirror of v10.3 *WalletState* for at least 4 quarters post-cutover; document the inverse mapping (overlay-key $u_{\mathrm{MA}} \to$ former *WalletState* field) explicitly.

F2, F5, F6 are the three that require external stakeholders before any line of code ships. F1, F3, F4, F7, F8 are internal engineering risks, manageable within the delivery programme.

9 Iteration Log

Twenty-seven independent agent invocations across eight rounds:

Round	Agents	Purpose	Key output
R1	grothendieck, noether, minsky, dirac, finops-architect, rosetta-cdm-engineer	Independent proposals	Convergence on three-sector model
R2	formalis, jane-street-cto, test-committee	Adversarial #1	Grothendieck / Dirac rejected; Minsky + Finops emerge

R3	geohot, karpathy, feynman, chris-lattner	Simplicity panel	*ProductTerms* split from *UnitStatus*; Option vs Monotone tension raised
R4	formalis, jane-street-cto, testcommittee	Adversarial #2	Option *and* Monotone (orthogonal); C1–C6 established
R5	gatheral (×2), finopsarchitect, correctnessarchitect	Test cases 1–4	HWM placement tension between managed-account and QIS; amendment two-track (C8)
R6	correctness-architect, formalis, jane-street-cto	Reconciliation	3 maps, not 4; mandate-as-unit legitimised; C1–C12 finalised
R7	testcommittee, correctnessarchitect, geohot	Pareto analysis	B Pareto-optimal on all three axes; convergence confirmed
R8	institutional-brake	Final gate	8 migration/governance risks logged; no design-level blocker

All agent reports are preserved in `/home/renaud/A61E33BB10/output/v10.3/StatesHome_work/R{1..8}_*.md`

10 Minimal Reference Implementation (≤ 50 lines)

```
from dataclasses import dataclass
from typing import Callable, Generic, Optional, TypeVar

U = TypeVar("U"); W = TypeVar("W"); T = TypeVar("T")

@dataclass(frozen=True)
class NonEmpty(Generic[T]):
    head: T; tail: tuple[T, ...] = ()
    def append(self, x: T) -> "NonEmpty[T]": return NonEmpty(self.head, self.tail
    + (x,))
    def current(self) -> T: return self.tail[-1] if self.tail else self.head

@dataclass(frozen=True)
class TermsVersion:
    fields: dict
    is_fungibility_preserving: Callable[["TermsVersion"], bool]

@dataclass(frozen=True)
class UnitStatus:
    lifecycle: str; last_px: Optional[float]; superseded_by: Optional[object] =
    None

@dataclass(frozen=True)
class PositionState:
    ac: float = 0.0; balance: float = 0.0; hwm: float = 0.0

FIELD_SPEC = {                                              # C11
    "ac":      {"conserved": True,  "handler": "settle"},
    "balance": {"conserved": True,  "handler": "transfer"},
    "hwm":     {"conserved": False, "monotone": True, "handler": "fee_crystallise"
    },
```

```
}

class Ledger:
    def __init__(self) -> None:
        self.PT: dict = {}                                  # ProductTerms
    C6, C7
        self.US: dict = {}                                  # UnitStatus
    C5
        self.PS: dict = {}                                  # PositionState
    C1

    def register(self, u, tv: TermsVersion, us: UnitStatus) -> None:
        if u in self.PT: raise ValueError("C10: re-registration")
        self.PT[u] = NonEmpty(tv); self.US[u] = us

    def position(self, w, u) -> Optional[PositionState]:    # C1 Option accessor
        return self.PS.get((w, u))

    def apply(self, delta: dict) -> None:                   # C3 atomic; C2
    checked; C11 handler-tagged
        for f, spec in FIELD_SPEC.items():
            if spec["conserved"] and sum(d.get(f, 0) for d in delta["rows"].values
    ()) != 0:
                raise ValueError(f"C2: {f} not conserved")
        for (w, u), diff in delta["rows"].items():
            old = self.PS.get((w, u), PositionState())
            self.PS[(w, u)] = PositionState(**{**old.__dict__, **diff})   #
    monotone: never deleted

    def amend(self, u, tv_new: TermsVersion, fresh: Callable[[], object]):  # C8
    two-track
        head = self.PT[u].current()
        if head.is_fungibility_preserving(tv_new):
            self.PT[u] = self.PT[u].append(tv_new); return u
        u2 = fresh()
        self.PT[u2] = NonEmpty(tv_new)
        self.US[u]  = UnitStatus(**{**self.US[u].__dict__, "superseded_by": u2})
        return u2
```

11 The One-Sentence Answer

> **Unit state lives in three places — immutable *ProductTerms*$[u]$, shared mutable *UnitStatus*$[u]$, and per-position *PositionState*$[w, u]$ — with no separate wallet-keyed state sector, because every economic per-wallet fact is naturally keyed by a mandate or strategy unit and collapses into *PositionState*$[w, u_{\text{MA}}]$.**

This is the Pareto-optimal schema on testability, correctness, and simplicity. It is the minimum basis of the problem, not a compromise. Ship B.

Document compiled 2026-04-22 from 27 adversarial agent rounds.
All supporting analyses preserved in `StatesHome_work/`.